

HURDLE 4

Dashboard & API Layer

Build a FastAPI backend and a Streamlit dashboard to visualise your data.

 35 Minutes

Objective

Create a REST API using FastAPI that exposes your carbon metrics and forecasting model. Then build a Streamlit dashboard that consumes the API and displays key sustainability KPIs with charts. Both should run locally before you deploy in Hurdle 5.

Background

A dashboard without a proper API layer is hard to maintain and impossible to scale. By separating the data/model layer (FastAPI) from the visualisation layer (Streamlit), you follow the same separation-of-concerns pattern used in production GreenOps platforms.

Tasks

Task A — FastAPI Backend

1. Create `api/main.py`. Start with the following boilerplate:

```
from fastapi import FastAPI
import pandas as pd
import joblib

app = FastAPI(title="GreenOps API")
df = pd.read_csv('data/cloud_usage_enriched.csv', parse_dates=['date'])
model = joblib.load('model/co2e_model.pkl')

@app.get('/health')
def health():
    return {"status": "ok"}
```

2. Implement all four endpoints listed in the table below.

Endpoint	Description
GET /health	Returns {"status": "ok"} — liveness probe
GET /metrics/summary	Returns total CO2e, cost, top emitting team and region
GET /metrics/daily	Returns daily CO2e array (date + value) for charting
GET /forecast	Loads the saved model and returns 30-day CO2e forecast

3. Run the API: `uvicorn api.main:app --reload`
4. Open `http://127.0.0.1:8000/docs` — verify all endpoints appear in the Swagger UI.
5. Test each endpoint and screenshot the response.

Task B — Streamlit Dashboard

6. Create `dashboard/app.py`.
7. Use `st.metric()` to display: Total CO2e (kg), Total Cost (USD), and Highest-Emission Team.
8. Use `st.line_chart()` or `plotly` to show the daily CO2e trend.
9. Add a 'Show Forecast' button that calls `GET /forecast` from the running API and plots the 30-day predicted emissions.
10. Run: `streamlit run dashboard/app.py`
11. Take a screenshot of the running dashboard in your browser.

Task C — Code Quality

12. Add a `requirements.txt`: `pip freeze > requirements.txt`
13. Add docstrings to each API endpoint.
14. Commit everything: 'feat: FastAPI backend and Streamlit dashboard'



Concept Check

- What is REST and why is it the standard for building APIs?
- What is the difference between a GET and a POST request? Which would you use to submit new billing data?
- Why run the API and dashboard as two separate processes rather than one combined script?



Self-Evaluation Checklist

☐	FastAPI app runs on port 8000 without errors
☐	All four endpoints respond correctly (verified in Swagger UI /docs)
☐	Streamlit dashboard displays CO2e metrics and daily trend chart
☐	Forecast button calls <code>/forecast</code> and plots the predicted values
☐	<code>requirements.txt</code> generated and committed
☐	Screenshots of Swagger UI and dashboard saved
☐	Concept check answered

» Proceed to Hurdle 5 once checklist is complete.